

# P&ID / Schema Unifilare Automatico — Architettura Drawing Director

## Stato decisionale

Questo documento fissa l'architettura da implementare nelle prossime sessioni. Le scelte qui riportate sono da considerare vincolanti salvo esplicita revisione.

L'obiettivo non è rendere il disegnatore deterministico “intelligente” tramite euristiche sempre più complesse. L'obiettivo è separare chiaramente: - comprensione e supervisione del layout; - ottimizzazione numerica; - esecuzione geometrica deterministica; - validazione finale oggettiva.

Il nuovo sistema deve quindi funzionare come un closed-loop supervisionato da un agente AI denominato **Drawing Director**.

---

## 1. Priorità immediata P0 — Correzione libreria simboli

Prima di lavorare sull'ottimizzazione del layout occorre eliminare gli errori simbolici.

### Problema emerso

Nel render corrente sono comparsi simboli errati, in particolare la valvola di ritegno. Questo indica che il mapping tra tipo logico del componente e simbolo grafico non è sufficientemente vincolato oppure che il renderer ha facoltà di selezionare/rappresentare simboli non canonici.

### Regola architetturale

Il Drawing Director non deve inventare né disegnare simboli.

Ogni componente deve essere definito da attributi strutturati, ad esempio:

```
component.type = CHECK_VALVE
component.standard = <standard_di_progetto>
component.symbol_id = CHECK_VALVE_001
```

Il renderer può utilizzare esclusivamente un simbolo presente nella libreria approvata e associato a `symbol_id`.

### Task P0

1. Audit completo della libreria simboli.
2. Identificazione di tutti i componenti con simbolo errato o ambiguo.

3. Correzione del mapping `component.type` -> `symbol_id`.
  4. Eliminazione di qualunque fallback grafico non esplicitamente autorizzato.
  5. Regression test automatico della libreria simboli.
  6. Test specifico della valvola di ritegno.
  7. Il Final Validator deve emettere FAIL se:
    - `symbol_id` non è autorizzato;
    - il simbolo renderizzato non corrisponde al tipo logico;
    - viene utilizzato un fallback non previsto.
- 

## 2. Problema architetturale attuale

Il flusso attuale è concettualmente:

```
Plant Graph
->
Deterministic Drawer
->
Disegno completo
->
AI Validator
->
PASS / FAIL / critica
```

Questa struttura è inefficiente perché il validatore interviene quando le decisioni geometriche errate sono già propagate nel disegno.

Esempi osservati: - tubazioni che avanzano verso il target e poi tornano indietro;  
- giri e deviazioni non necessari; - forte spazio morto tra gruppi funzionalmente correlati; - blocchi che potrebbero essere avvicinati senza violare i clearance;  
- routing complesso generato a valle di un posizionamento iniziale sbagliato; - ottimizzazione locale accettabile ma risultato globale mediocre.

La criticità principale non è il rendering. È la scelta del layout.

---

## 3. Nuova architettura — Drawing Director

Il validatore AI deve diventare un **supervisore attivo del disegnatore**.

Nuovo flusso:

```
Plant Graph
->
Initial Deterministic Layout
->
```

Partial / Current Render  
->  
Metric Extraction  
->  
AI Drawing Director  
->  
Structured Correction Commands  
->  
Deterministic Local Re-layout  
->  
Metric Recalculation  
->  
AI Drawing Director  
->  
...  
->  
Convergence / Stop Condition  
->  
Deterministic Final Validator  
->  
Export

## Principio fondamentale

Il Drawing Director: - NON deve disegnare; - NON deve produrre coordinate arbitrarie come unica fonte di verità; - NON deve sostituire il solver; - NON deve scegliere simboli graficamente; - NON deve valutare “a occhio” come unico criterio.

Il Drawing Director deve: 1. osservare il layout corrente; 2. leggere le metriche calcolate deterministicamente; 3. capire quali difetti globali stanno dominando; 4. tradurre direttive generali di progetto in correzioni operative; 5. modificare priorità, pesi, constraint e regioni da rilayouttare; 6. richiedere al motore deterministico una nuova soluzione; 7. confrontare il risultato prima/dopo; 8. decidere se accettare, correggere ancora o fare rollback.

---

## 4. Ruolo esatto del Drawing Director

Il Drawing Director è un agente di supervisione e planning.

Riceve almeno:

plant\_graph  
current\_layout  
current\_routes

current\_metrics  
current\_cost\_breakdown  
active\_constraints  
component\_metadata  
symbol\_validation\_status  
history\_of\_previous\_iterations  
general\_drawing\_directives

Restituisce esclusivamente decisioni strutturate.

Esempio concettuale:

```
{
  "priority": "reduce_functional_gap",
  "target_region": "ACS_BLOCK",
  "actions": [
    {
      "type": "MOVE_BLOCK",
      "block_id": "ACS_BLOCK",
      "direction": "LEFT",
      "max_delta": 320
    },
    {
      "type": "REROUTE_EDGE",
      "edge_id": "CP01.N03",
      "objective": "MINIMIZE_BACKTRACKING"
    },
    {
      "type": "CHANGE_WEIGHT",
      "term": "functional_affinity",
      "factor": 1.8
    },
    {
      "type": "ADD_CONSTRAINT",
      "constraint": "FORBID_HORIZONTAL_REVERSAL",
      "threshold": 120
    }
  ],
  "expected_effect": {
    "functional_gap": "decrease",
    "backtracking": "decrease",
    "pipe_length": "not_increase_more_than_3_percent"
  }
}
```

I comandi dovranno essere definiti tramite schema rigido.

Set minimo proposto: - MOVE\_BLOCK - SWAP\_BLOCKS - ALIGN\_BLOCKS -

COMPACT\_REGION - EXPAND\_REGION - REROUTE\_EDGE - REROUTE\_GROUP -  
CHANGE\_WEIGHT - ADD\_CONSTRAINT - REMOVE\_CONSTRAINT - LOCK\_REGION  
- UNLOCK\_REGION - REPLACE\_SYMBOL - REQUEST\_LOCAL\_OPTIMIZATION -  
ROLLBACK\_ITERATION

REPLACE\_SYMBOL non significa inventare il simbolo: deve solo selezionare un  
symbol\_id autorizzato dalla libreria.

---

## 5. Funzione di costo — principio

La qualità del disegno non deve essere giudicata esclusivamente dall'LLM.

Deve esistere una funzione di costo numerica, deterministica e scomponibile.

Forma generale:

$$J =$$
$$\begin{aligned} &wL * C\_length \\ &+ wB * C\_bends \\ &+ wX * C\_crossings \\ &+ wR * C\_backtracking \\ &+ wA * C\_area \\ &+ wD * C\_functional\_gap \\ &+ wO * C\_overlap \\ &+ wC * C\_clearance \\ &+ wS * C\_symbol \\ &+ wE * C\_editorial \\ &+ wM * C\_monotonicity \\ &+ wG * C\_graph\_semantics \end{aligned}$$

Dove ogni termine deve essere normalizzato, idealmente nell'intervallo  $[0,1]$   
oppure rispetto a una baseline.

La funzione globale deve restare deterministica.

L'AI non “decide il voto finale”. Decide come intervenire per ridurre J.

---

## 6. Termini della funzione di costo

### 6.1 Lunghezza tubazioni — C\_length

Misura la lunghezza totale delle tratte.

Obiettivo: - ridurre percorsi inutilmente lunghi; - evitare che uno spostamento  
migliori esteticamente il layout ma aumenti molto la quantità di tubazione.

Possibile normalizzazione:

$C\_length = actual\_total\_length / reference\_total\_length$

oppure rispetto alla shortest feasible routing length.

---

## 6.2 Curve / bend — **C\_bends**

Penalizza il numero di cambi di direzione.

Non tutte le curve devono pesare allo stesso modo.

Possibile schema: - cambio direzione necessario: costo base; - doppio cambio ravvicinato: costo superiore; - zig-zag: forte penalità; - deviazione non funzionale: penalità molto alta.

---

## 6.3 Crossing — **C\_crossings**

Penalizza: - attraversamenti tra tubazioni; - sovrapposizioni; - crossing in prossimità dei simboli; - crossing tra circuiti semanticamente differenti.

Alcuni crossing possono essere ammessi, ma devono essere costosi rispetto a una soluzione equivalente senza crossing.

---

## 6.4 Backtracking — **C\_backtracking**

È uno dei termini più importanti.

Definizione: una linea sta procedendo verso il target, poi compie un tratto che aumenta nuovamente la distanza dal target senza una necessità geometrica o topologica.

Questo cattura direttamente i tubi che “tornano indietro”.

Possibile misura: - somma delle distanze percorse nella direzione opposta al target; - numero di inversioni; - ampiezza dell’inversione.

Una inversione breve può avere costo moderato. Una inversione di centinaia di pixel deve avere costo elevato.

---

## 6.5 Area occupata / spazio morto — **C\_area**

Penalizza un layout eccessivamente espanso.

Non basta usare il bounding box globale: occorre distinguere tra: - spazio necessario per leggibilità; - clearance; - spazio morto privo di funzione.

Il caso osservato con il blocco ACS troppo lontano deve produrre una penalità significativa.

---

## 6.6 Functional affinity / functional gap — **C\_functional\_gap**

Questa metrica deve vincolare la distanza tra componenti o sottosistemi fortemente correlati.

Per due nodi o blocchi  $i, j$ :

$$C\_affinity(i,j) = A(i,j) * D(i,j)$$

dove: -  $A(i,j)$  = coefficiente di affinità funzionale; -  $D(i,j)$  = distanza geometrica.

L'affinità deve derivare dal Plant Graph e dalle relazioni impiantistiche.

Esempi: - generatore -> collettore immediato: affinità alta; - PdC -> accumulo collegato direttamente: affinità alta; - valvola -> componente che deve proteggere: alta; - elementi appartenenti a circuiti indipendenti: affinità bassa.

Obiettivo: se due blocchi sono fortemente collegati, lasciare grande spazio vuoto tra loro deve essere costoso.

---

## 6.7 Overlap — **C\_overlap**

Deve essere quasi un hard constraint.

Penalizza: - simbolo su simbolo; - testo su tubo; - testo su simbolo; - tubo dentro label; - tubo che attraversa elementi non attraversabili.

---

## 6.8 Clearance — **C\_clearance**

Distingue: - distanza minima obbligatoria; - distanza preferenziale; - distanza eccessiva.

Quindi non deve essere solo:

`distance >= minimum`

ma deve avere una zona preferenziale:

`too_close` -> penalità alta  
`preferred_band` -> penalità minima  
`too_far` -> penalità crescente quando esiste affinità funzionale

---

## 6.9 Simboli — `C_symbol`

Le violazioni simboliche devono essere gestite principalmente come hard constraint.

Esempio:

```
wrong_symbol = FAIL
missing_symbol = FAIL
unauthorized_symbol = FAIL
```

Il costo può essere usato durante l'iterazione, ma l'export finale non deve essere consentito con errori simbolici.

---

## 6.10 Editorial / leggibilità — `C_editorial`

Include: - label troppo vicine; - label allineate male; - nomenclature fuori regione; - frecce illeggibili; - densità eccessiva; - incoerenza della distribuzione visuale.

Questa è la componente in cui l'AI può fornire più valore interpretativo, ma il maggior numero possibile di aspetti deve essere misurato deterministicamente.

---

## 6.11 Directional monotonicity — `C_monotonicity`

Serve specificamente a evitare percorsi che si avvicinano al target e poi si allontanano.

Per ogni edge si definisce una direzione principale dal source al target.

Il routing deve essere preferibilmente monotono lungo tale direzione.

Le deviazioni sono ammesse solo per: - obstacle avoidance; - clearance; - constraint topologici; - passaggio ordinato in corridoi prefissati.

Una deviazione non giustificata genera costo.

---

## 6.12 Graph semantics — `C_graph_semantics`

Penalizza un layout visivamente possibile ma semanticamente sbagliato.

Esempi: - sequenza degli organi non rispettata; - valvola posizionata su ramo errato; - sensi di flusso incoerenti; - componente visivamente più vicino a un ramo a cui non appartiene; - routing che rende ambiguo il circuito.

Questo termine deve appoggiarsi al Plant Graph, non alla sola immagine.



---

## 7. Hard constraints vs soft costs

Non tutto deve essere ottimizzato con pesi.

### Hard constraints

Devono causare rifiuto della soluzione: - simbolo errato; - connessione topologica errata; - perdita di un componente; - sovrapposizione grave; - clearance minima violata; - sequenza funzionale alterata; - branch collegato al nodo sbagliato; - freccia di flusso contraria alla semantica; - geometria fuori foglio.

### Soft costs

Possono essere oggetto di trade-off: - lunghezza; - curve; - spazio morto; - distanza tra blocchi; - allineamento; - densità; - crossing ammessi; - preferenze estetiche/editoriali.

Il Drawing Director deve poter modificare i pesi solo dei soft costs.

Non deve mai “abbassare il peso” di un hard constraint per far passare una soluzione.

---

## 8. Come il Drawing Director ottimizza la funzione di costo

Questa è la parte centrale.

Il Drawing Director NON ottimizza direttamente J tramite testo libero.

Il loop deve essere:

1. Drawer genera layout L0
2. Metric Engine calcola:  
J0  
+ breakdown per termine
3. Director identifica i termini dominanti
4. Director seleziona interventi
5. Drawer genera L1
6. Metric Engine calcola J1
7. Confronto L1 vs L0
8. Se migliora -> accetta
9. Se peggiora -> rollback / cambio strategia
10. Ripeti fino a convergenza

## Esempio

Stato iniziale:

J = 0.74

|                |      |
|----------------|------|
| length         | 0.42 |
| bends          | 0.55 |
| crossings      | 0.10 |
| backtracking   | 0.88 |
| area           | 0.79 |
| functional_gap | 0.93 |
| overlap        | 0.05 |
| symbol         | FAIL |
| editorial      | 0.36 |
| monotonicity   | 0.82 |

Ordine operativo obbligatorio: 1. risolvere gli hard FAIL; 2. poi intervenire sui costi dominanti; 3. non ottimizzare dettagli marginali mentre esistono difetti macroscopici.

Quindi: - prima correggere **symbol**; - poi **functional\_gap**; - poi **backtracking** / **monotonicity**; - poi area e bend; - solo successivamente editorial fine-tuning.

---

## 9. Pesi dinamici ma controllati

I pesi non devono essere completamente liberi.

Si definiscono: - **base\_weight**; - **min\_weight**; - **max\_weight**; - eventuali profili di layout.

Esempio:

```
functional_gap:
  base = 1.0
  min = 0.8
  max = 3.0
```

```
backtracking:
  base = 1.5
  min = 1.2
  max = 4.0
```

Il Director può chiedere:

```
functional_gap x 1.5
backtracking x 2.0
```

ma solo entro i limiti previsti.

In questo modo l'LLM orienta l'ottimizzazione senza poter distruggere la funzione obiettivo.

---

## 10. Ottimizzazione locale prima della rigenerazione globale

Il sistema deve evitare di ridisegnare tutto ad ogni iterazione.

Ogni comando deve indicare una regione.

Esempio:

LOCK:

- PDC\_GROUP
- LEFT\_MANIFOLD

REOPTIMIZE:

- ACS\_BLOCK
- ROUTES\_TO\_ACS

Vantaggi: - stabilità; - meno regressioni; - convergenza più rapida; - migliore interpretabilità; - minore probabilità che una correzione a destra rovini la parte sinistra.

Il re-layout globale va utilizzato solo quando il Director determina che il problema dipende dalla struttura complessiva.

---

## 11. Validazione incrementale

Non aspettare il completamento del foglio.

Dopo ogni sottosistema:

```
generate region
-> validate topology
-> validate symbols
-> calculate local metrics
-> Director review
-> optimize
-> lock if acceptable
-> proceed
```

Esempio: 1. gruppo generatori; 2. collettori primari; 3. accumulo; 4. ACS; 5. secondari; 6. accessori; 7. label e rifiniture.

Una regione “locked” può essere riaperta solo esplicitamente.

---

## 12. Acceptance criteria per singola iterazione

Una proposta L1 può sostituire L0 solo se:

`hard_constraints(L1) = PASS`

e:

`J(L1) < J(L0) - epsilon`

oppure se il Director ha dichiarato esplicitamente un trade-off ammesso.

Esempio: aumentare la lunghezza tubazioni del 2% può essere accettabile se elimina 3 crossing e un grande backtracking.

Tale trade-off deve comunque essere verificato numericamente.

Il Director deve fornire l'effetto atteso prima della modifica e il sistema deve confrontarlo con l'effetto reale.

---

## 13. Anti-deriva del Drawing Director

Per evitare che una futura sessione o un diverso modello “reinventi” l'architettura:

### Regole non negoziabili

1. Il Plant Graph resta la fonte di verità funzionale.
  2. Il renderer/drawer resta deterministico.
  3. Il Drawing Director non genera geometria finale direttamente.
  4. La funzione di costo resta numerica e deterministica.
  5. L'AI può modificare solo pesi e constraint autorizzati.
  6. Gli hard constraints non sono negoziabili.
  7. Ogni iterazione deve avere metriche prima/dopo.
  8. Ogni regressione deve poter essere rollbackata.
  9. Le modifiche devono essere locali quando possibile.
  10. Il Final Validator resta separato dal Drawing Director.
  11. L'export è vietato in presenza di hard FAIL.
  12. I simboli provengono esclusivamente dalla libreria approvata.
-

## 14. Final Validator

Il Final Validator non deve essere lo stesso agente che dirige l'ottimizzazione.

Ruolo: - controllo finale indipendente; - deterministico per tutte le regole verificabili; - eventuale AI review aggiuntiva solo come quality gate non sostitutivo.

Controlli minimi: - Plant Graph rispettato; - component count corretto; - edge count / connectivity corretta; - direction corretta; - simboli corretti; - no hard overlap; - clearance minime; - niente elementi fuori foglio; - cartiglio e naming; - score globale entro soglia; - score dei singoli termini entro soglie specifiche.

Output:

PASS

oppure:

FAIL

- SYMBOL\_CHECK\_VALVE\_WRONG
  - BACKTRACKING\_EDGE\_CP01\_N03
  - FUNCTIONAL\_GAP\_ACS\_TOO\_HIGH
- 

## 15. Logging obbligatorio

Ogni iterazione deve essere tracciata.

```
iteration_id
layout_hash_before
metrics_before
director_actions
weights_before
weights_after
constraints_added
constraints_removed
layout_hash_after
metrics_after
accepted / rolled_back
reason
```

Questo log servirà per: - debug; - regression test; - valutare se il Director porta realmente miglioramenti; - evitare comportamenti non spiegabili.

---

## 16. Regression benchmark iniziale

Il disegno che ha evidenziato: - simbolo errato della valvola di ritegno; - tubazioni con ritorni; - giri inutili; - spazio morto centrale; - eccessiva distanza tra sottosistemi;

deve diventare il primo benchmark ufficiale del nuovo sistema.

Il test deve essere conservato con: - input Plant Graph; - layout iniziale; - metriche iniziali; - difetti attesi; - layout corretto; - metriche finali.

Acceptance target: - zero errori simbolici; - zero backtracking ingiustificati; - riduzione significativa dello spazio morto; - riduzione della distanza tra gruppi ad alta affinità; - nessun peggioramento topologico; - nessuna regressione di leggibilità.

---

## 17. Prossimi task — ordine vincolante

### P0 — Symbols

- audit libreria;
- fix valvola di ritegno;
- fix mapping;
- regression test simboli.

### P1 — Metric Engine

Implementare almeno: - pipe length; - bends; - crossings; - backtracking; - monotonicity; - area / dead space; - functional affinity; - overlap; - clearance.

### P2 — Structured Director API

Definire schema comandi: - move; - compact; - reroute; - weights; - constraints; - lock; - rollback.

### P3 — Drawing Director loop

Implementare:

draw -> measure -> critique -> command -> re-layout -> measure -> accept/rollback

### P4 — Local optimization / locking

Aggiungere: - region locking; - local re-layout; - incremental validation.

## P5 — Final Validator

Separare definitivamente: - Director; - optimizer; - final validation.

## P6 — Regression benchmark

Usare il caso attuale come test obbligatorio.

---

# 18. Criterio di successo dell'architettura

Il progetto è riuscito quando il sistema non si limita a produrre un disegno tecnicamente connesso, ma converge autonomamente verso una soluzione che:

- rispetta il Plant Graph;
- usa esclusivamente simboli corretti;
- riduce lunghezza e complessità inutile;
- evita backtracking;
- compatta i gruppi funzionalmente correlati;
- utilizza lo spazio in modo razionale;
- preserva clearance e leggibilità;
- produce risultati ripetibili;
- rende ogni decisione misurabile;
- consente rollback;
- non dipende dal gusto estemporaneo dell'LLM.

Il Drawing Director deve quindi essere considerato un **supervisore dell'ottimizzazione**, non un disegnatore generativo.